

Fair Signature Exchange

Hossein Hafezi², Aditi Partap⁴, Sourav Das³, and Joseph Bonneau^{1,2}

¹A16Z Crypto Research

²New York University (NYU)

³University of Illinois Urbana-Champaign (UIUC)

⁴Stanford University

`h.hafezi@nyu.edu, aditi712@stanford.edu, souravd2@illinois.edu, jcb@cs.nyu.edu`

Abstract. We introduce the concept of Fair Signature Exchange (FSE). FSE enables a client to obtain signatures on multiple messages in a *fair* manner: the client receives all signatures if and only if the signer receives an agreed-upon payment. We formalize security definitions for FSE and present a practical construction based on the Schnorr signature scheme, avoiding computationally expensive cryptographic primitives such as SNARKs. Our scheme imposes minimal overhead on the Schnorr signer and verifier, leaving the signature verification process unchanged from standard Schnorr signatures. Fairness is enforced using a blockchain as a trusted third party, while exchanging only a constant amount of information on-chain regardless of the number of signatures exchanged. We demonstrate how to construct a batch adaptor signature scheme using FSE, and our FSE construction based on Schnorr results in an efficient implementation of a batch Schnorr adaptor signature scheme for the discrete logarithm problem. We implemented our scheme to show that it has negligible overhead compared to standard Schnorr signatures. For instance, exchanging 2^{10} signatures on the Vesta curve takes approximately 80ms for the signer and 300ms for the verifier, with almost zero overhead for the signer and 2x overhead for the verifier compared to the original Schnorr protocol.

1 Introduction

Consider a scenario where a server holds a signing key and a client wishes to obtain signatures on a batch of messages in exchange for payment. For example, the client might be purchasing a set of certificates, or a set of one-time use tokens. In most existing applications, clients are forced to trust the server and pay upfront, receiving the signatures later. However, this model is unsuitable for decentralized environments where no party can be inherently trusted. Conversely, the server cannot send the signatures upfront, as the client's reliability to pay cannot be guaranteed.

In this paper, we introduce a new cryptographic primitive called *Fair Signature Exchange* (FSE). The primary goal of FSE is to enable a signer to *atomically* and *conditionally* exchange signatures on a batch of messages. Exchanging signatures is a special case of *fair exchange*, and it is well-established this is impossible for two parties to achieve without the help of a trusted third party (TTP) [9]. A natural idea is to instantiate the TTP using a blockchain. Tas et al. [13] proposed doing so for fair exchange of data as follows: the server first transmits encrypted data to the client, along with a commitment to the decryption key and a proof demonstrating that the commitment corresponds to the decryption key of the ciphertext. Subsequently, the client posts a conditional payment transaction on the blockchain, which is activated only if the commitment to the decryption key is correctly opened on the blockchain. This method is sound and requires only constant communication on the blockchain. However, it suffers from several drawbacks: the server-to-client communication overhead is high due to encryption overhead, and server-side computations are costly because the server must generate a Succinct Non-interactive Argument of Knowledge (SNARK) proof that the commitment opens to a valid decryption key corresponding to the encrypted message.

In this work, we propose a simpler primitive called *Fair Signature Exchange* (FSE). FSE provides a flexible framework that involves two parties, **signer** and **client**, both of whom have access to a TTP. The client possesses a set of messages $\{m_i\}$ and a public key pk , while the **signer** holds the corresponding secret key sk . The objective is for **client** to conditionally obtain signatures $\{\sigma_i\}$ on messages $\{m_i\}$ from **signer**, in

exchange for some agreed-upon payment. We present an FSE construction which minimizes communication using the blockchain while also reducing server-to-client communication. Importantly, our scheme does not rely on arguments of knowledge or SNARKs, significantly reducing the computational burden on the server. Our construction is based on the Schnorr signature scheme. We provide formal definitions for FSE akin to standard signature security definitions and prove the security of our construction. Additionally, we implement our scheme and show that it has zero overhead on the signer and only 2x overhead on the verifier compared to the original Schnorr protocol and importantly, the verification procedure remains the same.

FSE can be used to build the batch version of *adaptor signatures* for the discrete logarithm relation, which facilitates the atomic exchange of a single signature and a discrete logarithm of a public group element. An adaptor signature is a cryptographic primitive that efficiently facilitates such atomic exchange using a blockchain as a trusted third party (we elaborate more on adaptor signatures in §2). Adaptor signatures are widely used in decentralized finance (DeFi), payment channels (batch atomic swaps) and multi-chain bridges; however, unlike FSE, they are limited to exchanging one signature (or payment) at a time, which becomes impractical when dealing with multiple accounts or transactions simultaneously.

2 Related work

The closest to our work are the fair-data exchange (FDE) construction [13], and adaptor signatures [5]. Our work is inspired by FDE but for signatures. We can exchange data significantly more efficiently by focusing on exchanging signatures.

Fair data exchange (FDE). In the FDE setting, the client holds a commitment to some data, which the server possesses. The concept of fairness is defined in two parts: the client cannot obtain the data without paying the server (*Server Fairness*) and the server cannot receive payment without revealing the data to the client (*Client Fairness*). Tas et al., in [13] presents two concrete constructions of FDE. In both constructions, the server encrypts the data with a random key. It sends the ciphertexts to the client, along with a commitment to the key, and a zero-knowledge proof that the ciphertexts encrypt the correct data with the key and commitment to the key opens to the key. The server and client then exchange the key for some agreed-upon payment via the trusted third party. However, their constructions are expensive since they require data encryption and arguments of knowledge. Akin to FDE, in FSE, **signer** has some secret key sk and **client** has the corresponding public input pk , but this secret value sk is never revealed to the client. Furthermore, unlike the setting in [13], FSE must address the risk of a faulty client attempting to learn a signature on one message by querying signatures for other messages. This was not a concern in the server fairness definition of [13]. Additionally, in our scheme, we only rely on the hardness of discrete logarithm, without requiring expensive SNARKs, as in [13].

Adaptor signatures. An adaptor signature is a cryptographic protocol that extends standard digital signatures to enable conditional commitments. In this scheme, the signer generates an incomplete signature with respect to a NP statement Y . Any client with the corresponding witness y can obtain the full signature. Adaptor signatures are extractable, meaning that the signer can compute the witness y using the full signature and the corresponding incomplete signature. This mechanism is widely used in trustless atomic swaps within DeFi [6] and payment channels [4, 10, 8].

Adaptor signatures enable trustless cross-chain exchanges by embedding a cryptographic secret into partially signed transactions, ensuring atomicity. Consider Alice (owning Bitcoin) and Bob (owning Ethereum) who agree to exchange assets using a shared secret x and its hash $H(x)$. Alice locks 1 BTC in a Bitcoin hash time-locked contract (HTLC), allowing Bob to claim it only by revealing x within a deadline. Instead of fully signing the transaction, Alice provides an adaptor signature, which can only be finalized by revealing x . Concurrently, Bob locks 10 ETH in an Ethereum HTLC, allowing Alice to claim it with x . This setup ensures the exchange cannot be completed unless both parties act. Bob finalizes Alice’s adaptor signature to claim the Bitcoin, which reveals x in the process. Alice then uses x to unlock the Ethereum on Bob’s HTLC. If either party fails to act before the deadlines, the locked funds are returned to their respective owners, preserving security. This mechanism ensures atomicity: either both exchanges are completed, or neither

happens. One attractive feature of adaptor signatures is that they do not require the blockchain to support smart contracts, only require the support of HTLC.

As we mentioned earlier, FSE can be used to extend adaptor signatures for the discrete logarithm relation to the batch setting, wherein the client can get multiple signatures in exchange for a single secret discrete logarithm value. Batching is particularly useful for complex DeFi scenarios e.g. batch atomic swaps. The key efficiency of our scheme lies in using a single secret value to complete all transactions (signatures), regardless of the batch size. The signer can extract the secret witness when any one of the signatures from the batch is posted onchain by the client. In contrast, classic adaptor signature schemes would require a unique secret for each transaction, resulting in a linear increase in blockchain communications.

Functional adaptor signatures. Functional Adaptor Signatures (FAS) [14] is an extension of adaptor signatures that enables a server and a client to fairly exchange a signature for a function evaluation $f(x)$ where x is some secret value. In this protocol, the server learns nothing about x beyond the value of $f(x)$, and can compute $f(x)$ if and only if the client gets a valid signature. On a similar vein as FDE, the client encrypts x using functional encryption to get c_x . The client and server then use adaptor signatures to exchange the functional secret key for f for a signature. The server can then decrypt c_x using the functional secret key to obtain $f(x)$.

3 Preliminaries

Notations. For any integer n , we use $[n]$ to denote the ordered set $\{1, 2, \dots, n\}$. For any non-empty set S , we let $s \leftarrow \$ S$ indicate that s is sampled uniformly at random from S , and we use $|S|$ to denote the size of S . We use λ to denote the security parameter. We denote negligible functions with $\text{negl}(\lambda)$. A machine is called probabilistic polynomial time (PPT) if it is a probabilistic algorithm that runs in $\text{poly}(\lambda)$ time. All algorithms are probabilistic unless stated otherwise. By $y \leftarrow A(x_1, \dots, x_n)$, we denote the operation of running algorithm A on inputs $(x_1, \dots, x_n)$ with uniformly random coins and letting y denote the output. If A has oracle access to some algorithm E , we write $y \leftarrow A^{E(\cdot)}(x_1, \dots, x_n)$.

Computational assumptions. Let GGen be a group generation algorithm that on input 1^λ outputs the description of a prime order group $\mathbb{G}$. The description contains the prime order p , a generator $g \in \mathbb{G}$, and a description of the group operation. We will use the multiplicative notation for the group operation. Our protocol assumes the standard discrete logarithm (DL) assumption in group $\mathbb{G}$, which we formally define in Definition 6. We denote a random oracle using $\mathcal{H}$ that we assume is randomly sampled from random oracle space $\mathcal{H}$.

Digital signature. It is a cryptographic protocol consisting of four algorithms $\Pi = (\text{Setup}, \text{Gen}, \text{Sign}, \text{Verify})$, defined as follows:

- $\text{Setup}(1^\lambda)$: The setup algorithm that takes as input a security parameter λ and outputs system parameters pp . These parameters include the description of the message space and the key space.
- $\text{Gen}(\text{pp})$: The key generation algorithm that takes the system parameters pp and outputs a pair of keys (sk, pk) , where sk is the secret (signing) key and pk is the public (verification) key.
- $\text{Sign}(\text{sk}, m)$: The signing algorithm that takes the secret key sk and a message m , and outputs a signature σ .
- $\text{Verify}(\text{pk}, m, \sigma)$: The verification algorithm is a deterministic algorithm that takes the public key pk , a message m , and a signature σ , and outputs a bit $b \in \{0, 1\}$. The output $b = 1$ indicates that the signature is valid, and $b = 0$ indicates that it is invalid.

A digital signature scheme is secure if it satisfies correctness and *strong* unforgeability as defined below:

Unforgeability $_{\mathcal{A}}^{\Pi}(1^{\lambda})$	Oracle $\text{SIGN}(m)$
1 : $\text{pp} \leftarrow \Pi.\text{Setup}(1^{\lambda})$	1 : $\sigma \leftarrow \Pi.\text{Sign}(\text{sk}, m)$
2 : $(\text{pk}, \text{sk}) \leftarrow \Pi.\text{KGen}(\text{pp}); \mathcal{Q} := \emptyset$	2 : $\mathcal{Q} = \mathcal{Q} \cup \{(m, \sigma)\}$
3 : $(m^*, \sigma^*) \leftarrow \mathcal{A}^{\text{SIGN}(\cdot)}(\text{pp}, \text{pk})$	3 : return σ
4 : return $(m^*, \sigma^*) \notin \mathcal{Q} \wedge \Pi.\text{Verify}(\text{pk}, m^*, \sigma^*) = 1$	

Fig. 1: Unforgeability game for signature scheme Π

Definition 1 (Signature Correctness). *Signature scheme $\Pi = (\text{Setup}, \text{Gen}, \text{Sign}, \text{Verify})$ is correct, if the following holds:*

$$\Pr \left[b = 1 \mid \begin{array}{l} \text{pp} \leftarrow \Pi.\text{Setup}(1^{\lambda}) \\ (\text{sk}, \text{pk}) \leftarrow \Pi.\text{Gen}(\text{pp}) \\ \sigma \leftarrow \Pi.\text{Sign}(\text{sk}, m) \\ b \leftarrow \Pi.\text{Verify}(\text{pk}, m, \sigma) \end{array} \right] = 1.$$

Definition 2 (Signature Strong Unforgeability). *A signature scheme $\Pi = (\text{Setup}, \text{Gen}, \text{Sign}, \text{Verify})$ has strong existential unforgeability under chosen-message attack, if any PPT adversary $\mathcal{A}$ wins the game defined in Figure 1 with negligible probability.*

The Schnorr signature scheme [12]. Let $(\mathbb{G}, p, g) \leftarrow \text{GGen}(1^{\lambda})$. Let $\text{H}_{\text{Sig}} : \mathbb{G}^2 \times \mathcal{M} \rightarrow \mathbb{Z}_p$ be a hash function modelled as a random oracle, where $\mathcal{M}$ is the message space. The signing key $\text{sk} \in \mathbb{Z}_p$ is a random field element, and $\text{pk} := g^{\text{sk}} \in \mathbb{G}$ is the corresponding public verification key. The signature σ on a message m is then $(R, s) \in \mathbb{G} \times \mathbb{Z}_p$. To validate a signature $\sigma = (R, s)$ on a message m , a validator first computes $c := \text{H}_{\text{Sig}}(R, \text{pk}, m)$ and checks that $g^s = R \cdot \text{pk}^c$. Schnorr signatures have been proven strongly unforgeable in the random oracle model [12, 2].

4 Fair Signature Exchange

4.1 Definitions

A *Fair Signature Exchange* (FSE) protocol is a protocol between client and signer involving a transparent payment environment $\mathbf{E}$ as a trusted third party (TTP) that holds money under addresses belonging to the other parties. The TTP can transfer money from one party's address to another but requires a transaction authorizing the transfer with the sender's signature. It is transparent in the sense that any message sent to $\mathbf{E}$ eventually becomes visible to all other parties. The FSE protocol $\text{FSE}[\Pi]$ initialized with a signature scheme Π , consists of PPT algorithms (Setup , KeyGen , PSign , PVerify , Execute , Recover). Both the client and the signer parties are assumed to have access to the functionalities of Π . For the sake of simplicity from now on, we write FSE instead of $\text{FSE}[\Pi]$.

- $\text{FSE.Setup}(1^{\lambda}) \rightarrow \text{pp}$ is a randomized algorithm that outputs the public parameters for the system (e.g., the description of appropriate spaces). All the following algorithms and protocols implicitly take the pp as input.
- $\text{FSE.KeyGen}(\text{pp}) \rightarrow (\text{pk}, \text{sk})$ is a randomized algorithm that samples a secret signing key sk and computes the corresponding public key pk .
- $\text{FSE.PSign}(\text{sk}, \{m_i\}_{i \in [n]}) \rightarrow (\{\tilde{\sigma}_i\}_{i \in [n]}, \text{aux}, \text{com}_k)$ is a randomized algorithm that takes as input the secret key sk and a batch of messages $\{m_i\}_{i \in [n]} \in \mathcal{M}^n$. It samples an encryption key $k \leftarrow_{\$} \mathcal{K}$, computes partial (masked) signatures $\{\tilde{\sigma}_i\}_{i \in [n]}$ along with auxiliary data aux and a commitment com_k to the key. Here $\mathcal{M}$ and $\mathcal{K}$ denote the message space and the key space of Π respectively. We also use $\text{FSE.PSign}(\text{sk}, \{m_i\}_{i \in [n]}; k)$ to denote the above process.

- $\text{FSE.PVerify}(\text{pk}, \{m_i\}_{i \in [n]}, \{\tilde{\sigma}_i\}_{i \in [n]}, \text{aux}, \text{com}_k) \rightarrow \{0, 1\}$ is a deterministic algorithm that on input public key pk , partial signatures $\{\tilde{\sigma}_i\}_{i \in [n]}$ auxiliary data aux and a key commitment com_k , checks whether the partial signatures are valid.
- $\text{FSE.Execute}(\langle \text{client}(\text{com}_k, \text{tok}) \leftrightarrow \text{E} \leftrightarrow \text{signer}(k) \rangle)$. client and signer communicate independently with the TTP environment E , at the end of which, the client receives the opening k to commitment com_k , and signer receives some token tok . This interaction can be realized with the following algorithms, wherein $\text{FSE}^{(\text{E})}$ indicates the algorithm having access to the environment E :

$$\begin{aligned} \text{st}_{\text{E},0} &\leftarrow \text{FSE}^{(\text{E})}.\text{Init}() \\ \text{st}_{\text{E},1} &\leftarrow \text{FSE}^{(\text{E})}.\text{ExeClient}_0(\text{com}_k, \text{tok}) \\ (\text{tok}_s, \text{st}_{\text{E},2}) &\leftarrow \text{FSE}^{(\text{E})}.\text{ExeSigner}_0(k) \end{aligned}$$

In essence, the interaction on E is successful if the signer receives the payment, i.e. $\text{tok}_s \neq \perp$.

- $\text{FSE.Recover}(\{\tilde{\sigma}_i\}_{i \in [n]}, \text{aux}, k) \rightarrow \{\sigma_i\}_{i \in [n]}$ is a deterministic algorithm that, on input partial signatures $\{\tilde{\sigma}_i\}_{i \in [n]}$, auxiliary data aux and the decryption key k , outputs signatures $\{\sigma_i\}_{i \in [n]}$.

We require the FSE protocol to meet the following properties: *completeness*, *signer fairness*, *client fairness*. Completeness ensures that if both parties honestly follow the protocol, then in the end, the signer receives the tokens and the client receives signatures. The signer fairness property ensures that a malicious client cannot obtain more valid signatures than it has paid for. Lastly, the client fairness property guarantees that a dishonest signer would not be paid without committing to valid signatures and subsequently revealing them to the client. We formalize these properties as follows.

Definition 3 (Completeness). *We say that a fair signature exchange scheme $\text{FSE}[\Pi]$ is complete if the following probability is 1 for any polynomial batch size $n = n(\lambda)$, and any set of polynomially long messages $\{m_i\}_{i \in [n]} \in \mathcal{M}^n$:*

$$\Pr \left[\begin{array}{l} b = 1 \wedge \text{tok}_s \neq \perp \wedge \\ \forall i \in [n] : \\ \Pi.\text{Verify}(\text{pk}, m_i, \sigma_i) = 1 \end{array} \middle| \begin{array}{l} \text{pp} \leftarrow \text{FSE.Setup}(1^\lambda) \\ (\text{sk}, \text{pk}) \leftarrow \text{FSE.KeyGen}(); k \leftarrow \$_K \\ (\{\tilde{\sigma}_i\}_{i \in [n]}, \text{aux}, \text{com}_k) \leftarrow \text{FSE.PSign}(\text{sk}, \{m_i\}_{i \in [n]}; k) \\ b := \text{FSE.PVerify}(\text{pk}, \{\tilde{\sigma}_i\}_{i \in [n]}, \text{aux}, \text{com}_k) \\ \{\sigma_i\}_{i \in [n]} := \text{FSE.Recover}(\{\tilde{\sigma}_i\}_{i \in [n]}, \text{aux}, k) \\ \text{st}_{\text{E},0} \leftarrow \text{FSE}^{(\text{E})}.\text{Init}() \\ \text{st}_{\text{E},1} \leftarrow \text{FSE}^{(\text{E})}.\text{ExeClient}_0(\text{com}_k, \text{tok}) \\ (\text{tok}_s, \text{st}_{\text{E},2}) \leftarrow \text{FSE}^{(\text{E})}.\text{ExeSigner}_0(k) \end{array} \right]$$

Definition 4 (Signer Fairness). *We say the scheme FSE has signer-fairness if for all PPT adversaries $\mathcal{A}$, the following quantity is negligible in λ :*

$$\text{Adv}_{\mathcal{A}, \text{FSE}[\Pi]}^{\text{sf}}(\lambda) = \Pr[\text{SignerFairness}_{\mathcal{A}}^{\text{FSE}[\Pi]}(1^\lambda) = 1]$$

where the signer fairness game is as defined in Figure 2.

Definition 5 (Client Fairness). *We say the scheme FSE has client-fairness if any PPT adversary $\mathcal{A}$ wins the client fairness game in Figure 3 game with negligible probability.*

5 Design

We now describe our construction of the FSE protocol. For simplicity, we assume client has a single message m and seeks to obtain a signature for it from signer . We consider the case of a vector of messages later in the section.

Game $\text{SignerFairness}_{\mathcal{A}}^{\text{FSE}[\Pi]}(1^\lambda)$	Oracle $\text{SIGN}(\{m_i\}_{i \in [n]})$
1 : $\text{pp} \leftarrow \text{FSE.Setup}(1^\lambda)$	1 : $c \leftarrow c + 1$
2 : $(\text{pk}, \text{sk}) \leftarrow \text{FSE.KGen}(\text{pp}, 1^\lambda)$	2 : $k \leftarrow \mathcal{K}; \mathcal{K}[c] := k$
3 : $\mathcal{S} := \phi, \mathcal{K} := \phi, \mathcal{N} := \phi, c = 0, t = 0$	3 : $\text{msg} \leftarrow \text{FSE.PSign}(\text{sk}, \{m_i\}_{i \in [n]}; k)$
4 : $\{(m_i^*, \sigma_i^*)\}_{i \in t'} \leftarrow \mathcal{A}^{\text{SIGN}(\cdot), \text{EXEC}(\cdot)}(\text{pp}, \text{pk})$	4 : $\mathcal{S} := \mathcal{S} \cup \{c\}; \mathcal{N}[c] = n$
5 : return $t < t' \wedge$	5 : return (c, msg)
6 : $\forall i \in [t'] : \Pi.\text{Verify}(\text{pk}, m_i^*, \sigma_i^*) = 1 \wedge$	Oracle $\text{EXEC}(j, \text{com}_k)$
7 : $\forall i \neq j \in [t'] : m_i^* \neq m_j^*$	1 : if $j \notin \mathcal{S} \vee \text{com}_k \neq \text{com}(\mathcal{K}[j])$ return $\perp$
	2 : $k_j := \mathcal{K}[j]; n_j := \mathcal{N}[j]$
	3 : $t = t + n_j$
	4 : return k_j

Fig. 2: Signer fairness game $\text{SignerFairness}_{\mathcal{A}}^{\text{FSE}[\Pi]}(1^\lambda)$ for a fair signature exchange protocol $\text{FSE}[\Pi]$. Note that we consider a strong adversary that can start multiple signing sessions, and additionally, run `Execute` multiple times for each session.

Game $\text{ClientFairness}_{\mathcal{A}}^{\text{FSE}[\Pi]}(1^\lambda)$	Oracle $\text{EXEC}(j, k, \text{tok})$
1 : $\text{pp} \leftarrow \text{FSE.Setup}(1^\lambda)$	1 : if $j \notin \mathcal{S}$ return $\perp$
2 : $(\text{pk}, \text{st}) \leftarrow \mathcal{A}(\text{pp})$	2 : $(\{m_i\}_{i \in [n]}, \{\tilde{\sigma}_i\}_{i \in [n]}, \text{aux}, \text{com}_k) := \mathcal{I}[j]$
3 : $\mathcal{B} := \phi, \mathcal{I} := \phi, \mathcal{F} := \phi, \mathcal{S} := \phi, c \leftarrow 0$	3 : $\text{st}_{\text{E},0} \leftarrow \text{FSE.Init}()$
4 : $(\text{st}) \leftarrow \mathcal{A}^{\text{VERIFY}(\cdot), \text{EXEC}(\cdot)}(\text{st})$	4 : $\text{st}_{\text{E},1} \leftarrow \text{FSE.ExeClient}_0(\text{com}_k, \text{tok})$
5 : if $\exists i \in [c]$ s.t. $\mathcal{B}[i] = 1 \wedge$	5 : $(\text{tok}_s, \text{st}_{\text{E},2}) \leftarrow \text{FSE.ExeSigner}_0(k)$
6 : $\exists (m, \sigma) \in \mathcal{F}[i] : \Pi.\text{Verify}(\text{pk}, m, \sigma) = 0 :$	6 : $\{\sigma_i\}_{i \in [n]} \leftarrow \text{FSE.Recover}(\{\tilde{\sigma}_i\}_{i \in [n]}, \text{aux}, k)$
7 : return 1	7 : $\mathcal{S} \leftarrow \mathcal{S} \cup \{j\}$
8 : else return 0	8 : if $\text{tok}_s = \perp$ return $\perp$
Oracle $\text{VERIFY}(\{m_i\}_{i \in [n]}, (\{\tilde{\sigma}_i\}, \text{aux}, \text{com}_k))$	9 : $\mathcal{F}[j] \leftarrow \{(m_i, \sigma_i)\}_{i \in [n]}$
1 : $c \leftarrow c + 1$	10 : return $\{\sigma_i\}_{i \in [n]}$
2 : $\mathcal{B}[c] \leftarrow \text{FSE.PVerify}(\text{pk}, \{\tilde{\sigma}_i\}_{i \in [n]}, \text{aux}, \text{com}_k)$	
3 : $\mathcal{I}[c] \leftarrow (\{m_i\}_{i \in [n]}, \{\tilde{\sigma}_i\}_{i \in [n]}, \text{aux}, \text{com}_k)$	
4 : $\mathcal{S} \leftarrow \mathcal{S} \cup \{c\}$	

Fig. 3: Client fairness game $\text{ClientFairness}_{\mathcal{A}}^{\text{FSE}[\Pi]}(1^\lambda)$ for a fair signature exchange protocol FSE .

We start by describing the following basic construction: on receiving the message m , the **signer** first generates a Schnorr signature $\sigma = (R, s)$ on the message, where $R = g^r$ is a masked randomness generated using a uniformly random value r , and s is a field element s.t. $g^s = R \cdot \text{pk}^{\text{H}_{\text{Sig}}(R, \text{pk}, m)}$. Initially, the **signer** sends only the masked randomness R to client, along with a commitment com to the signature s . **signer** will reveal the value s only after the client pays **signer** – this step can be executed atomically using `E`. Although this approach is elegant, several important issues must be resolved, which we list below:

1. client needs a proof that the commitment com actually opens up to the correct value s , that is, that $\sigma = (R, s)$ is a valid signature for the message of client. This is important so that client does not have to trust **signer** blindly.

FSE.Setup (1^λ) 1 : $(\mathbb{G}, p, g) \leftarrow \text{GGen}(1^\lambda)$ 2 : $H \leftarrow \mathcal{H}$ 3 : return $\text{pp} := (\mathbb{G}, p, g, H)$ FSE.KeyGen () 1 : $\text{sk} \leftarrow \mathbb{Z}_p$ 2 : return $(\text{sk}, \text{pk} := g^x)$ FSE.PVerify ($\text{pk}, \{m_i\}_{i \in [n]}, \{\tilde{\sigma}_i\}_{i \in [n]}, \text{aux}, \text{com}_k$) 1 : Parse $\{R_i\}_{i \in [n]} \leftarrow \text{aux}$ 2 : $\forall i \in [n] : c_i := H(R_i, m_i)$ 3 : return true if $\forall i \in [n] :$ 4 : $\text{com}_i := R_i \cdot \text{pk}^{c_i}$ 5 : $g^{2\tilde{\sigma}_i} \stackrel{?}{=} \text{com}_k \cdot \text{com}_i$	FSE.PSign ($\text{sk}, \{m_i\}_{i \in [n]}$) 1 : $k \leftarrow \mathbb{Z}_p; \text{com}_k := g^k$ 2 : $\forall i \in [n] :$ 3 : $r_i \leftarrow \mathbb{Z}_p; R_i := g^{r_i}$ 4 : $c_i := H(R_i, m_i)$ 5 : $s_i := r_i + c_i \cdot \text{sk}$ 6 : $\tilde{\sigma}_i := \frac{k + s_i}{2}$ 7 : return $(\{\tilde{\sigma}_i\}_{i \in [n]}, \text{aux} := \{R_i\}_{i \in [n]}, \text{com}_k)$ FSE.Recover ($\{\tilde{\sigma}_i\}_{i \in [n]}, \text{aux}, k$) 1 : Parse $\{R_i\}_{i \in [n]} \leftarrow \text{aux}$ 2 : $\forall i \in [n] : s_i := 2 \cdot \tilde{\sigma}_i - k$ 3 : return $\{\sigma_i := (R_i, s_i)\}_{i \in [n]}$
--	--

Fig. 4: FSE protocol for fair signature exchange of Schnorr signatures

2. The final step, wherein **signer** reveals the commitment opening s on $\mathbf{E}$ and $\mathbf{E}$ subsequently transfers the payment to **signer**, will be very expensive when running the protocol for a vector of messages, because the communication on $\mathbf{E}$ will grow linearly with the number of messages to be signed. This is not ideal, since communication on $\mathbf{E}$ is much more expensive than direct communication between **signer** and **client**. Ideally, we would like the communication to remain constant and independent of the number of messages.

We address the first issue by leveraging the inherent structure of Schnorr signatures. Specifically, recall that the Schnorr verification equation is $g^s \stackrel{?}{=} R \cdot \text{pk}^c$ where $c = H_{\text{sig}}(\text{pk}, R, m)$. Hence, the **signer** can simply send R along with g^s as a commitment to s . **client** can easily verify this commitment by just checking the Schnorr verification equation. Importantly, **signer** does not need to send any additional data to validate these commitments. This eliminates the need for an argument of knowledge, making our protocol significantly more efficient than a general-purpose Fair Data Exchange (FDE).

To address the second issue, consider the scenario in which parties need to exchange signatures on n messages. In a naive approach, **client** and **signer** would send the commitments $\{g^{s_i}\}$ and corresponding openings $\{s_i\}$ on $\mathbf{E}$, resulting in linear communication in n . Instead, we propose a modified signing protocol, wherein the **signer** samples a symmetric encryption key k , and sends encrypted values $\{\tilde{\sigma}_i\}$ derived from signatures $\{s_i\}$ along with a commitment to the key k (Figure 5). More formally, **signer** computes the partial signatures $\tilde{\sigma}_i = (k + s_i)/2$ as illustrated in Figure 4. Later we prove $\tilde{\sigma}_i$ effectively *hides* the underlying signature s_i . **signer** sends these along with a binding commitment to the key, $\text{com}_k = g^k$, to **client**. To exchange a batch of n signatures, **client** and **signer** only need to transmit the decryption key k and its commitment com_k through $\mathbf{E}$, reducing the communication complexity in $\mathbf{E}$ from linear in the batch size to constant. Once **client** obtains k , they can easily recover the original signatures s_i from the masked values $\tilde{\sigma}_i$. Figure 5 depicts the masking process – since the **client** only sees the partial signatures, we can prove that the **client** cannot compute the signatures without the decryption key k via $\mathbf{E}$, based on the discrete log assumption.

An overview of the whole interaction between parties **signer**, **client** and $\mathbf{E}$ can be seen in Figure 6. Figure 4 provides a formal overview of our proposed scheme. Additionally, Figure 7 presents the interaction via $\mathbf{E}$ to exchange the key k for tokens.

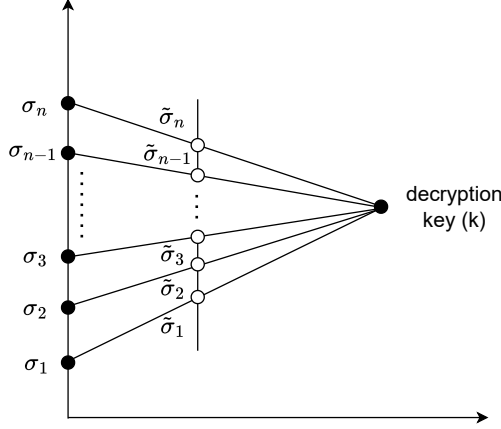


Fig. 5: Let σ_i represent the field elements of the Schnorr signatures requested by the client. k denotes the symmetric encryption key, which is uniformly randomly sampled by the server. The expression $\tilde{\sigma}_i = \frac{\sigma_i + k}{2}$ represents the “encryption” of σ_i under the key k . The client and server now only need to exchange k via the transparent environment. Crucially, we can securely use the same key k for all n signatures since the signature values σ_i are random and independent, from the view of the client.

5.1 Analysis

Theorem 1. *The fair signature exchange scheme FSE described in Figure 4 satisfies completeness.*

Proof. Let $\text{pp} = (\mathbb{G}, p, g, H)$ be the parameters output by the setup algorithm $\text{FSE.Setup}(1^\lambda)$ and $\{m_i\}_{i \in [n]} \in \mathcal{M}^n$ be any set of messages that $n \in \text{poly}(\lambda)$. Suppose (sk, pk) is the key pair output by $\text{FSE.Gen}()$, where $\text{pk} = g^{\text{sk}}$. Let $k \leftarrow \mathbb{Z}_p$, and define

$$(\{\tilde{\sigma}_i\}_{i \in [n]}, \{R_i\}_{i \in [n]}, \text{com}_k) \leftarrow \text{FSE.PSign}(\text{sk}, \{m_i\}_{i \in [n]}; k)$$

Where $R_i = g^{r_i}$ for random values r_i , $\tilde{\sigma}_i = \frac{k + r_i + H(R_i, m_i) \cdot \text{sk}}{2}$, and $\text{com}_k = g^k$. First, we assert that

$$\text{FSE.PVerify}(\text{pk}, \{m_i\}_{i \in [n]}, \{\tilde{\sigma}_i\}_{i \in [n]}, \{R_i\}_{i \in [n]}, \text{com}_k) = 1$$

Which holds if and only if $g^{2\tilde{\sigma}_i} \stackrel{?}{=} \text{com}_k \cdot R_i \cdot \text{pk}^{c_i}$. Expanding this:

$$g^{2\tilde{\sigma}_i} = g^{k + r_i + H(R_i, m_i) \cdot \text{sk}} = g^k \cdot g^{r_i} \cdot (g^{\text{sk}})^{H(R_i, m_i)} = \text{com}_k \cdot R_i \cdot \text{pk}^{c_i}$$

Secondly, let $\{\sigma_i\}_{i \in [n]} := \text{FSE.Recover}(\{\tilde{\sigma}_i\}_{i \in [n]}, \{R_i\}_{i \in [n]}, k)$, which implies that $\sigma_i = (2 \cdot \tilde{\sigma}_i - k, R_i)$. Therefore, the following holds:

$$\text{II.Verify}(\text{pk}, m_i, \sigma_i) = 1 \iff g^{2 \cdot \tilde{\sigma}_i - k} \stackrel{?}{=} R_i \cdot \text{pk}^{c_i}$$

Furthermore, we can verify:

$$g^{2 \cdot \tilde{\sigma}_i - k} = g^{r_i + H(R_i, m_i) \cdot \text{sk}} = g^{r_i} \cdot (g^{\text{sk}})^{H(R_i, m_i)} = R_i \cdot \text{pk}^{c_i}$$

Finally since $\text{com}_k = g^k$, according to the definition of transparent environment $\mathbf{E}$, client having tokens tok , receives k and $\text{tok}_s \neq \perp$. Thus, all three conditions hold with a probability of 1.

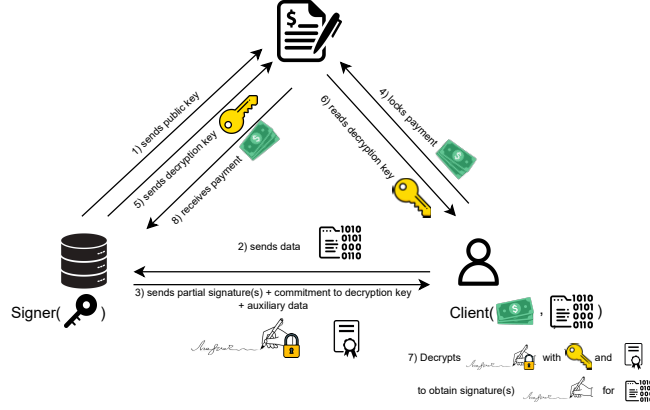


Fig. 6: Overview of the interaction between signer, client and the environment E , in a Fair signature exchange (FSE) protocol.

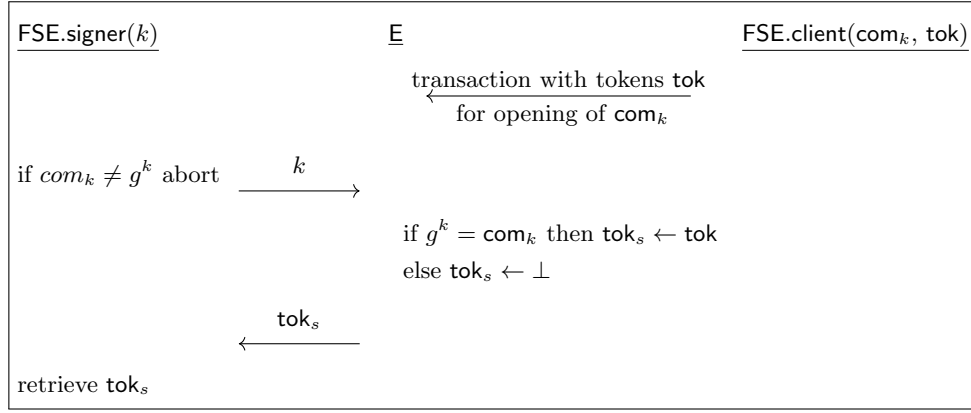


Fig. 7: Interaction of signer and client with the transparent environment E . This can be implemented on Ethereum using a smart contract, and on Bitcoin using adaptor signatures, similar to [13].

Theorem 2. *The fair signature exchange scheme FSE described in Figure 4 satisfies signer fairness. In fact, for every PPT adversary $\mathcal{A}$, there exist PPT adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that,*

$$\text{Adv}_{\mathcal{A}}^{\text{sf}}(\lambda) \leq q_s \cdot \left(\text{Adv}_{\mathcal{B}_1}^{\text{dl}}(\lambda) + \frac{n \cdot q_H}{p} \right) + \text{Adv}_{\mathcal{B}_2, \text{Sch}}^{\text{suf}}(\lambda)$$

where $n = n(\lambda)$, $q_s = q_s(\lambda)$ and $q_H = q_H(\lambda)$ are upper bounds on the batch size, number of signing oracle queries, and number of random oracle queries made by $\mathcal{A}$ respectively.

Proof. We prove the security signer fairness game based on the *strong unforgeability* of Schnorr signatures and the DL assumption, as defined in Definitions 2 and 6, respectively. To provide intuition, assume that the adversary $\mathcal{A}$ outputs n_1 valid message/signature pairs while recovering only n_2 messages through FSE.Recover , where $n_2 < n_1$. The adversary must have obtained a signature on a message m^* in one of the following ways:

- If $\mathcal{A}$ has not called FSE.PSign on m^* , we formally construct another adversary capable of breaking the unforgeability of the Schnorr signature scheme.
- If $\mathcal{A}$ has called FSE.PSign on m^* but has not called FSE.Recover on m^* , this implies that for a random g^k , computing the signature on m^* enables $\mathcal{A}$ to compute k . In this case, we formally construct an adversary capable of breaking the DL assumption.

Given the security of the Schnorr signature scheme (strong unforgeability) and the DL assumption, We conclude that, except with negligible probability, $\mathcal{A}$ cannot produce a signature for a message that has not been queried either to the signing oracle or to FSE.Recover . This proves the signer fairness game. The formal proof can be found in Appendix A.1.

Theorem 3. *The fair signature exchange scheme FSE described in Figure 4 satisfies client fairness. In fact, every PPT adversary $\mathcal{A}$ wins $\text{ClientFairness}_{\mathcal{A}}^{\text{FSE}[\Pi]}(1^\lambda)$ for scheme Figure 4 with an advantage of 0.*

Proof. Intuitively, given $\text{com}_k = g^k$, client can verify the correctness of partial signatures by checking:

$$\begin{aligned} \text{FSE.PVerify}(\{m_i\}_{i \in [n]}, \{\tilde{\sigma}_i\}_{i \in [n]}, \text{aux}, \text{com}_k) &= 1 \\ \iff g^{2\tilde{\sigma}_i} &= \text{com}_k \cdot R_i \cdot \text{pk}^{c_i}. \end{aligned}$$

By trusting $\mathbf{E}$, client pays signer if and only if it receives the key k such that $\text{com}_k = g^k$. This ensures that client makes the payment if and only if it can compute the correct signatures. The formal proof can be found in Appendix A.2.

5.2 Constructing Batch Adaptor Signatures from FSE

Adapting the definition of adaptor signatures from [5], we define the notation of *batch adaptor signature scheme* $\text{BAS}_{\Pi, \text{Rel}}$ as an extension of adaptor signature scheme to sign multiple messages simultaneously, with respect to a signature scheme Π and a hard relation Rel is defined as a tuple of four PPT algorithms:

- $\{\tilde{\sigma}_i\}_{i \in [n]} \leftarrow \text{pBSign}(\text{sk}, \{m_i\}_{i \in [n]}, Y)$: The pre-signing algorithm takes as input the secret key for Π , a batch of messages, and a statement $Y \in L_{\text{Rel}}$, and outputs partial signatures on all the messages.
- $b \leftarrow \text{pVerify}(\text{pk}, \{m_i\}_{i \in [n]}, Y, \{\tilde{\sigma}_i\}_{i \in [n]})$: The pre-verification algorithm is a deterministic algorithm which takes as input the public key for Π , a batch of messages, a statement Y and corresponding partial signatures $\{\tilde{\sigma}_i\}_{i \in [n]}$, outputs a bit denoting whether or not all the partial signatures are valid.
- $\{\sigma_i\}_{i \in [n]} \leftarrow \text{Adapt}(\text{pk}, \{\tilde{\sigma}_i\}, y)$: The adapting algorithm takes as input a batch of partial signatures, a witness y for the statement Y , and outputs full signatures.
- $y \leftarrow \text{Extract}(\text{vk}, \tilde{\sigma}_i, \sigma_i, Y)$: The extracting algorithm takes as input a partial signature, a corresponding full signature, a statement Y , and outputs a witness y such that $(Y, y) \in \text{Rel}$ or $\perp$.

Similar to adaptor signatures [5], a batch adaptor signature scheme must satisfy the following properties:

- *Partial signature correctness.* This means that if the client and the server are both honest, then the client successfully gets valid signatures on all the messages queried, and the server learns the witnesses of all the corresponding instances.
- *Partial signature adaptability.* Informally, this means that if the partial signatures satisfy verification, i.e. BVerify outputs one, then the client gets valid signatures after running Adapt .
- *Unforgeability.* A malicious client should not be able to forge a signature.
- *Witness extractability.* This guarantees that a malicious client cannot use a partial signature for a message m , with respect to a statement Y to produce a valid signature σ without revealing a witness for Y .

We formally define properties above in Appendix B. We now discuss how we can construct a Schnorr-signature-based batch adaptor scheme for the discrete log relation from our FSE construction. The core idea, is to think of the encryption key k as the client's discrete log witness, and the same witness k is used to compute partial signatures for the whole batch of messages. More formally, when given the discrete log instance Y as input, the signer samples $\{r_i\}$ values as in standard Schnorr signing, then sets $\hat{R}_i$ to be g^{r_i}/Y for all i . The challenge values c_i 's are computed using $\hat{R}_i$ instead of R_i , and then the signer simply returns $\frac{r_i + c_i \cdot \text{sk}}{2}$. It is easy to see that the FSE.PVerify algorithm can directly be used to verify the partial signatures. The FSE.Recover algorithm can be used to adapt the partial signatures to get full signatures. Lastly, given

BAS.pBSign(sk, {m _i }, Y)	BAS.BVerify(pk, {m _i } _{i∈[n]} , Y, {σ̃ _i } _{i∈[n]})
1 : $\forall i \in [n] :$	1 : Parse (s _i , R' _i) $\leftarrow \tilde{\sigma}_i \forall i \in [n]$
2 : $r_i \leftarrow \mathbb{Z}_p; R'_i := g^{r_i} / Y$	2 : $\forall i \in [n] : c_i := H(R'_i, m_i)$
3 : $c_i := H(R'_i, m_i)$	3 : return true if $\forall i \in [n] :$
4 : $s_i := \frac{r_i + c_i \cdot \text{sk}}{2}$	4 : $\text{com}_i := R'_i \cdot \text{pk}^{c_i}$
5 : return ({(s _i , R' _i)} _{i∈[n]})	5 : $g^{2s_i} \stackrel{?}{=} Y \cdot \text{com}_i$
BAS.Extract(pk, σ̃ _i , σ _i , Y)	BAS.Adapt(pk, {σ̃ _i } _{i∈[n]} , y)
1 : return 2 · s _i − s' _i	1 : $\forall i \in [n] : s'_i := 2 \cdot s_i - y$

Fig. 8: The batch adaptor signature scheme BAS based on Schnorr signatures and the discrete log relation, constructed from our FSE scheme.

any full signature $\sigma_i = (\hat{R}_i, s_i)$ and the corresponding partial signature $\hat{\sigma}_i$, the signer can easily compute the discrete log as $2 \cdot \hat{\sigma}_i - s_i$. Figure 8 formally presents the batch adaptor construction.

Analysis. Partial signature correctness for our construction is straightforward. Adaptability of the scheme directly follows from client fairness of the FSE scheme. Unforgeability can be proven by relying on signer fairness of FSE – the client cannot forge more signatures. Lastly, witness extractability holds based on the unforgeability of the underlying Schnorr signature scheme, similar to the proof in [3]. We elaborate more on this in Appendix B.

6 Implementation and Evaluation

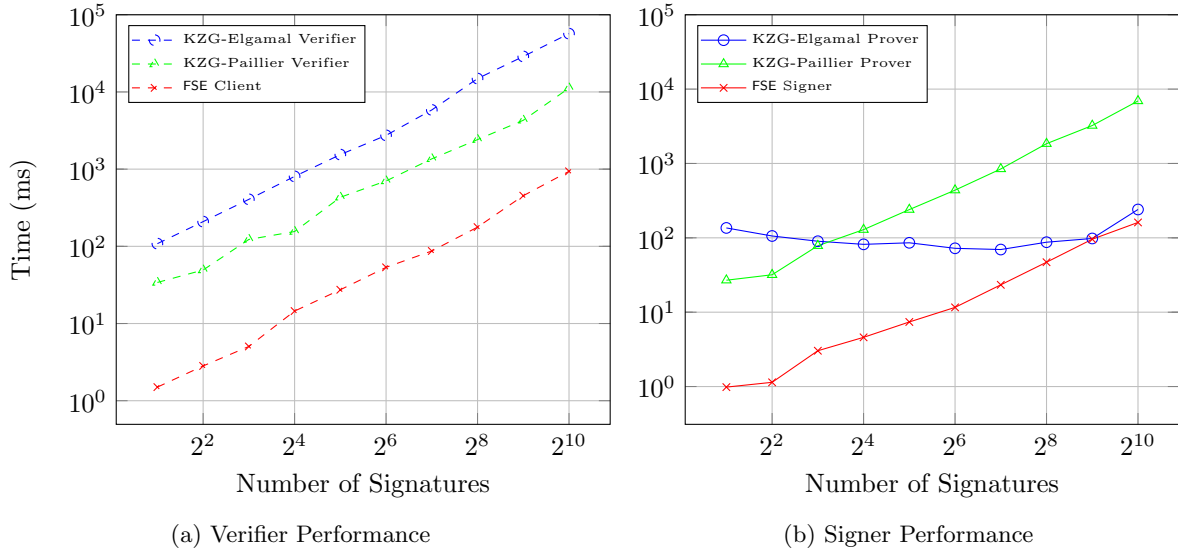


Fig. 9: Comparison between our FSE scheme and two schemes in [13], all instantiated with BLS12-381. (a) Verifier Comparison, (b) Prover Comparison. The verifier in our FSE scheme is about 10x more efficient than KZG-Paillier verifier and 100x more efficient than KZG-Elgamal. Our prover is about 10x more efficient than the KZG-Paillier prover, and 1.03 – 100x faster relative to the KZG-Elgamal prover.

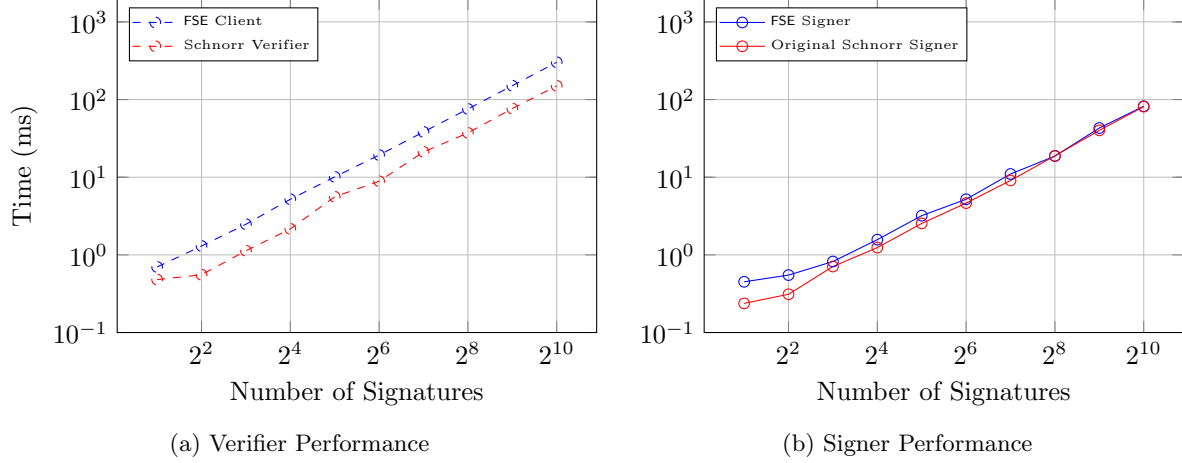


Fig. 10: Comparison of Verifier and Signer performance between FSE scheme and Original Schnorr, both instantiated with Vesta curve. FSE has almost no overhead on the signer but has a 2x overhead on the verifier (client) compared to the original Schnorr scheme.

To assess the performance of our protocol, we implemented it in Rust using the Arkworks framework [1]. Our source code is publicly available.* All experiments were conducted on a consumer-grade PC equipped with an Intel i5-7200U CPU (2 cores) and 8 GB of RAM. Our simple baseline for measurement is the original Schnorr protocol. To measure the computation of the **signer**, we consider the running time of FSE.PSign algorithm in Figure 4, and for the **client**, we measure the running time of the algorithms FSE.PVerify and FSE.Recover. We find that, compared to the original Schnorr protocol, our protocol has only 25% overhead for the signer for $n > 8$ signatures (and upto $0.9x$ for smaller values of n); and under 2x overhead for the verifier (Figure 10). We evaluate our scheme on two curves: BLS12-381 (pairing-friendly) and Vesta (non-pairing-friendly). Notably, initiating our scheme on Vesta yields a 2-3x performance improvement compared to BLS12-381, which is advantageous as we do not require pairing.

Computation comparison to general-purpose FDE [13]. We benchmark our protocol against the state-of-the-art FDE schemes presented in [13], which we refer to as KZG-Paillier and KZG-Elgamal (Figure 9). For this comparison, we assume that the transferred data consists of a vector of field elements. A single Schnorr signature comprises a randomly chosen group element R and a secret field element s . We assume s is transmitted through FDE, allowing us to equate a signature with a field element and assume R is transmitted in plaintext. Similar to the original experiment in [13], we measure the time required to generate proofs by the prover and verify proofs by the verifier. The FDE scheme employs KZG polynomial commitment on the pairing-friendly BLS12-381 curve, introducing additional overhead. It is important to note that the FDE scheme includes additional costly operations, such as data encryption and KZG commitment generation, which are not accounted for in their performance evaluation but are irrelevant to our protocol and, therefore, not included in our comparison. For the sake of comparison, we use BLS12-381 to initialize our scheme too. As seen in Figure 9, the signer in our FSE protocol is almost 10x more efficient than the KZG-Paillier prover; relative to the KZG-Elgamal prover, our signer is $1.03 - 100x$ faster. The verifier in our FSE scheme is about 10x more efficient than the KZG-Paillier verifier and 100x more efficient than the verifier in KZG-Elgamal. Hence, our FSE scheme in general is orders of magnitude more efficient than the general purpose FDE constructions in [13].

Communication comparison to [13]. Communication using blockchains is extremely expensive. At the time of writing, publishing 1MB of data on the Ethereum blockchain costs approximately \$17,000 USD. Therefore, minimizing on-chain communication is crucial. Our protocol, like [13], operates FSE for n signa-

*Anonymized repository for peer review: https://anonymous.4open.science/r/fair_schnorr_signature_exchange-148F

tures with constant on-chain communication: only a single group element (com_k) and one scalar field element (k) are published on $\mathbf{E}$, totalling just 136 bytes with BLS12-381. Unlike [13], our protocol also minimizes off-chain communication. To exchange n signatures (beyond the messages to be signed), the **signer** and **client** only exchange n field elements $\{\tilde{\sigma}_i\}_{i \in [n]}$ and $n + 1$ group elements: $\{R_i\}_{i \in [n]}$ along with com_k . This contrasts sharply with [13], which incurs significant bandwidth overheads, with constants 10x and 50x larger for KZG-Elgamal and KZG-Paillier, respectively, relative to the data size.

7 Conclusion

Fair Signature Exchange (FSE) is a novel cryptographic primitive inspired by Fair Data Exchange (FDE). It generalizes the concept of adaptor signatures by extending their functionality to support batching, enabling conditional and atomic exchange of multiple signatures in a single operation. FSE has significant applications in decentralized finance (DeFi), including batch atomic swaps and bridges. FSE extends the definition of adaptor signatures to consider the blockchain in the security definitions as well as batching. In this work, we formally define FSE and provide comprehensive security definitions as well as a construction based on Schnorr signatures. Our work includes rigorous proofs of security, showing that FSE achieves its goals while minimizing communication with the trusted third party (e.g., blockchain) and does not rely on heavy cryptographic primitives like SNARKs. This makes FSE a lightweight and efficient solution for fair signature exchanges in decentralized ecosystems.

7.1 Future work

Fair blind signature exchange. A particularly intriguing primitive is the fair blind signature exchange, which finds practical applications in areas such as coin shuffling and the fair exchange of privacy passes. A key question is whether it is possible to construct a fair blind signature exchange akin to FSE that introduces minimal overhead to the original blind signature scheme and avoids the use of arguments of knowledge.

Fair proof exchange. Proof systems are useful tools that help outsource computation trustlessly with tremendous applications such as roll-ups. One interesting question is how to construct efficient *fair proof exchange* which can be the building block for a *proof marketplace*. Distributed proof systems [11, 7, 15] involve a coordinator that partitions a computation into smaller chunks. These chunks are then assigned to worker nodes which compute proofs for their respective chunks. Subsequently, the coordinator aggregates the individual chunk proofs to produce a proof for the entire computation. Fair proof exchange mechanisms can incentivize worker nodes for their contributions, while ensuring they are paid if they provide the agreed-upon proofs.

Acknowledgments. We thank Ali Yazdizadeh for his assistance with a technical issue. The initial stages of this project were conducted while Sourav Das and Aditi Partap were at a16z Crypto Research. This work was supported by a16z Crypto. Joseph Bonneau was also supported by DARPA Agreement HR00112020022 and NSF Grant CNS-2239975. The views and conclusions contained in this material are those of the authors and do not necessarily reflect the official policies or endorsements of the United States Government, DARPA, a16z Crypto, or any other supporting organization.

References

1. arkworks contributors. **arkworks** zksnark ecosystem, 2022.
2. Dan Boneh and Victor Shoup. A graduate course in applied cryptography, 2023. Cryptobook.
3. Andreas Erwig, Sebastian Faust, Kristina Hostáková, Monosij Maitra, and Siavash Riahi. Two-party adaptor signatures from identification schemes. Cryptology ePrint Archive, Paper 2021/150, 2021.
4. Muhammed F. Esgin, Oguzhan Ersoy, and Zekeriya Erkin. Post-quantum adaptor signatures and payment channel networks. Cryptology ePrint Archive, Paper 2020/845, 2020.
5. Paul Gerhart, Dominique Schröder, Pratik Soni, and Sri AravindaKrishnan Thyagarajan. Foundations of adaptor signatures. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 161–189. Springer, 2024.
6. Philipp Hoenisch, Subhra Mazumdar, Pedro Moreno-Sanchez, and Sushmita Ruj. LightSwap: An atomic swap does not require timeouts at both blockchains. Cryptology ePrint Archive, Paper 2022/1650, 2022.
7. Tianyi Liu, Tiancheng Xie, Jiaheng Zhang, Dawn Song, and Yupeng Zhang. Pianist: Scalable zkRollups via fully distributed zero-knowledge proofs. Cryptology ePrint Archive, Paper 2023/1271, 2023.
8. Arash Mirzaei, Amin Sakzad, Jiangshan Yu, and Ron Steinfeld. Daric: A storage efficient payment channel with penalization mechanism. Cryptology ePrint Archive, Paper 2022/1295, 2022.
9. Henning Pagnia and Felix C. Gartner Darmstadt. On the impossibility of fair exchange without a trusted third party. 1999.
10. Siavash Riahi and Orfeas Stefanos Thyfronitis Litos. Bitcoin clique: Channel-free off-chain payments using two-shot adaptor signatures. Cryptology ePrint Archive, Paper 2024/025, 2024.
11. Michael Rosenberg, Tushar Mopuri, Hossein Hafezi, Ian Miers, and Pratyush Mishra. Hekaton: Horizontally-scalable zkSNARKs via proof aggregation. Cryptology ePrint Archive, Paper 2024/1208, 2024.
12. Claus-Peter Schnorr. Efficient identification and signatures for smart cards. In Gilles Brassard, editor, *CRYPTO’89*, volume 435 of *LNCS*, pages 239–252, Santa Barbara, CA, USA, August 20–24, 1990. Springer, New York, USA.
13. Ertem Nusret Tas, István András Seres, Yinuo Zhang, Márk Melczer, Mahimna Kelkar, Joseph Bonneau, and Valeria Nikolaenko. Atomic and fair data exchange via blockchain. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, 2024.
14. Nikhil Vanjani, Pratik Soni, and Sri AravindaKrishnan Thyagarajan. Functional adaptor signatures: Beyond all-or-nothing blockchain-based payments. Cryptology ePrint Archive, Paper 2024/1523, 2024.
15. Wenhao Wang, Fangyan Shi, Dani Vilardell, and Fan Zhang. Cirrus: Performant and accountable distributed SNARK. Cryptology ePrint Archive, Paper 2024/1873, 2024.

A Deferred Proofs

We now prove that our proposed scheme in Figure 4 satisfies signer-fairness and client-fairness.

A.1 Server fairness

Definition 6 (Discrete Logarithm Assumption). *We say the discrete logarithm assumption (DL) holds with respect to group generator $\text{GGen}(1^\lambda)$ if for any PPT adversary $\mathcal{A}$, given $(\mathbb{G}, p, g) \leftarrow \text{GGen}(1^\lambda)$ and $x \leftarrow \mathbb{Z}_p$, $\mathcal{A}(\mathbb{G}, p, g, g^x)$ outputs x with negligible probability.*

Proof of Theorem 2. Let $\mathcal{L} = \{m_i^*, \sigma_i^*\}_{i \in [t']}$ denote the list of signatures returned by $\mathcal{A}$ in the signer fairness game. Let $\sigma_i^* = (R_i^*, s_i^*)$ be the signatures returned by $\mathcal{A}$, for all $i \in [t']$. Let $\mathbf{M}$ denote the list of messages for which $\mathcal{A}$ queried the SIGN oracle. Let $\mathbf{R} : \mathcal{M} \rightarrow \mathbb{G}$ be a map storing the R_i values output by the challenger in response to a SIGN query by $\mathcal{A}$. Let $\mathbf{B} : \mathcal{M} \rightarrow \mathbb{N}$ be a map storing the batch number (or session id) corresponding to the messages for which the SIGN oracle was called. Lastly, let $\mathbf{X}$ be a list of session ids for which the EXECUTE is called, and it does not return $\perp$.

Let $\mathbf{E}_1$ be the event that there exists a message - signature pair $(m^*, \sigma^* = (R^*, s^*))$ such that (a) either $m^* \notin \mathbf{M}$, meaning that the adversary never called the SIGN oracle with m^* as one of the inputs, (b) or, $m^* \in \mathbf{M}$, $R^* \neq \mathbf{R}[m^*]$, and $\mathbf{B}[m^*]$ is not in $\mathbf{X}$. Additionally, let $\mathbf{E}_2$ denote the event that there exists a message - signature pair $(m^*, \sigma^* = (R^*, s^*))$ such that $m^* \in \mathbf{M}$, the corresponding nonce value R^* is equal to $\mathbf{R}(m^*)$, and, $\mathbf{B}[m^*]$ is not in $\mathbf{X}$.

We note that in the event $\neg \mathbf{E}_1 \wedge \neg \mathbf{E}_2$, the number t' of signatures output by $\mathcal{A}$ cannot be greater than t because for all (m, σ) pairs output by the adversary, both the SIGN and EXEC oracles were called and successfully run. Hence, the adversary can win the signer fairness game only if either $\mathbf{E}_1$ or $\mathbf{E}_2$ occurs. This implies that,

$$\Pr[\text{SignerFairness}_{\mathcal{A}}^{\text{FSE}[\Pi]}(1^\lambda) = 1] \quad (1)$$

$$= \Pr[\text{SignerFairness}_{\mathcal{A}}^{\text{FSE}[\Pi]}(1^\lambda) \wedge \mathbf{E}_1] \quad (2)$$

$$+ \Pr[\text{SignerFairness}_{\mathcal{A}}^{\text{FSE}[\Pi]}(1^\lambda) \wedge \mathbf{E}_2] \quad (3)$$

Lemmas 1 and 2 below prove the theorem.

Lemma 1. *There exists a PPT adversary $\mathcal{B}_1$ such that,*

$$\Pr[\text{SignerFairness}_{\mathcal{A}}^{\text{FSE}[\Pi]}(1^\lambda) = 1 \wedge \mathbf{E}_1] \leq \text{Adv}_{\mathcal{B}_1, \text{Sch}}^{\text{unf}}(\lambda)$$

Proof. We construct an adversary $\mathcal{B}_1$ for the strong unforgeability of Schnorr as follows. It receives $\text{pp} \leftarrow (\mathbb{G}, g, p, H)$, pk from its challenger and simulates the SignerFairness game to $\mathcal{A}$. It first sends pp , pk to $\mathcal{A}$. It maintains maps $\mathbf{M}$, $\mathbf{R}$ and $\mathbf{B}$ as described above – they are all initialized as $\perp$. It then responds to the oracle queries as follows:

- **SIGN**($\{m_i\}_{i \in [n]}$). As in the signer fairness game, $\mathcal{B}_1$ increments the counter c by one, and samples a random key $k \leftarrow \mathbb{Z}_p$. It then queries its unforgeability challenger for signatures on m_i for all $i \in [n]$. Let $\{(R_i, s_i)\}_{i \in [n]}$ be the challenger's response. It computes $\text{com}_k \leftarrow g^k$, and $\tilde{\sigma}_i = (k + s_i)/2$ for all $i \in [n]$. It stores k and n in the maps $\mathcal{K}$ and $\mathcal{N}$ respectively. Additionally, it sets $\mathbf{M} \leftarrow \mathbf{M} \cup \{m_i\}_{i \in [n]}$, $\mathbf{R}[m_i] \leftarrow R_i$ and $\mathbf{B}[m_i] \leftarrow c$ for all $i \in [n]$. It then returns $\{\tilde{\sigma}_i\}_{i \in [n]}$, com_k and $\text{aux} = \{R_i\}_{i \in [n]}$ to $\mathcal{A}$.
- **EXEC**(j, com_k). $\mathcal{B}_1$ aborts if $j \notin \mathcal{K}$ or if $\text{com}_k \neq \text{com}(\mathcal{K}[j])$, as in the signer fairness game. Otherwise, it increments t by n , adds j to $\mathbf{X}$ and outputs $\mathcal{K}[j]$.
- **H**(R, m). It queries its challenger for $H(R, m)$ and forwards the response to $\mathcal{A}$.

Eventually, $\mathcal{A}$ outputs a list of message - signature pairs, $\mathcal{L} = \{m_i^*, \sigma_i^*\}_{i \in [t']}$. $\mathcal{B}_1$ finds a message-signature pair $(m^*, (R^*, s^*)) \in \mathcal{L}$ such that, either $m^* \notin \mathbf{M}$ or, $m^* \in \mathbf{M}$ but $R^* \neq \mathbf{R}[m^*]$ and $\mathbf{B}[m^*]$ is not in $\mathbf{X}$. It aborts if it cannot find any such message. Otherwise, it returns $(m^*, (R^*, s^*))$ to its challenger. Now, we argue that if $\mathcal{A}$ wins the signer fairness game, and if $\mathbf{E}_1$ occurs, then $\mathcal{B}_1$ also wins its unforgeability game. This is because, conditioned on $\mathbf{E}_1$, $\mathcal{B}_1$ will not abort because there exists a message in $\mathcal{L}$ that satisfies all the conditions listed above. Additionally, if $m^* \notin \mathbf{M}$, then, $\mathcal{B}_1$ never queried its challenger for a signature on m^* . Otherwise, if $m^* \in \mathbf{M}$ but $R^* \neq \mathbf{R}[m^*]$, then, the signature (R^*, s^*) on m^* output by $\mathcal{A}$ is different from the signature $(\mathbf{R}[m^*], \cdot)$ output by the unforgeability challenger. In both cases, $\mathcal{B}$ successfully outputs a valid forgery. This proves the lemma.

Lemma 2. *There exists a PPT adversary $\mathcal{B}_2$ such that,*

$$\frac{1}{q_s} \cdot \Pr[\text{SignerFairness}_{\mathcal{A}}^{\text{FSE}[\Pi]}(1^\lambda) = 1 \wedge \mathbf{E}_2] - \frac{n \cdot q_H}{p} \leq \text{Adv}_{\mathcal{B}_2}^{\text{dl}}(\lambda)$$

where $q_s = q_s(\lambda)$, $n = n(\lambda)$ and $q_H = q_H(\lambda)$ are upper bounds on the number of SIGN queries, the batch size, and the number of random oracle queries issued by $\mathcal{A}$ respectively.

Proof. We construct a discrete log adversary $\mathcal{B}_2$ as follows. It gets as input $(\mathbb{G}, g, h, p)$, where p is the order of the group $\mathbb{G}$, and g is a generator. It plays the role of challenger to $\mathcal{A}$ in the SignerFairness game as follows. First, it samples $\text{sk} \leftarrow \mathbb{Z}_p$ and sets $\text{pk} \leftarrow g^{\text{sk}}$. It then sends $\text{pp} = (\mathbb{G}, g, p), \text{pk}$ to $\mathcal{A}$. To respond to oracle queries, $\mathcal{B}_2$ maintains some metadata. Specifically, it stores a map $H : \mathbb{G} \times \mathcal{M} \rightarrow \mathbb{Z}_p$ to handle random oracle queries. It initializes another map $\mathbf{A} : \mathcal{M} \rightarrow \mathbb{Z}_p$ to store $\tilde{\sigma}$ values used for responding to SIGN queries. It also maintains $\mathbf{M}$, $\mathbf{R}$ and $\mathbf{B}$ as described above. Additionally, let q_s be an upper bound on the number of signing oracle queries issued by $\mathcal{A}$. $\mathcal{B}_2$ samples $i^* \leftarrow \mathbb{Z}_p$, as its guess for the signing session containing a forgery message. We now describe how $\mathcal{B}_2$ responds to oracle queries:

- $H(R, m)$. If $(R, m) \in H$, then simply output $H(R, m)$. Otherwise, it samples $c \leftarrow \mathbb{Z}_p$, sets $H(R, m) \leftarrow c$ and returns c to $\mathcal{A}$.
- $\text{SIGN}(\{m_i\}_{i \in [n]})$. $\mathcal{B}$ increments c as in the game. It sets $\mathbf{M} \leftarrow \mathbf{M} \cup \{m_i\}_{i \in [n]}$ and $\mathbf{B}[m_i] \leftarrow c$ for all $i \in [n]$.
 - If $c \neq i^*$, $\mathcal{B}_2$ honestly runs the signature protocol. More formally, for each $i \in [n]$, it samples $r_i \leftarrow \mathbb{Z}_p$, computes $R_i \leftarrow g^{r_i}$. It queries $c_i \leftarrow H(R_i, m_i)$ for all $i \in [n]$, and then computes $s_i \leftarrow r_i + c_i \cdot \text{sk}$. It samples $k_c \leftarrow \mathbb{Z}_p$, computes $\text{com}_k \leftarrow g^{k_c}$ and stores $\mathcal{K}[c] \leftarrow k_c$. It computes $\tilde{\sigma}_i \leftarrow (k_c + s_i)/2$ and $\text{com}_i \leftarrow g^{s_i}$ for all $i \in [n]$. It then sets $\mathbf{R}[m_i] \leftarrow R_i$ for all $i \in [n]$, and sends $\{\tilde{\sigma}_i\}_{i \in [n]}$, $\text{aux} = \{R_i\}_{i \in [n]}$ and com_k to $\mathcal{A}$.
 - If $c = i^*$, $\mathcal{B}$ sets $\text{com}_k^* \leftarrow h$.
 - * It samples $\tilde{\sigma}_1, \dots, \tilde{\sigma}_n \leftarrow \mathbb{Z}_p$.
 - * It then computes $\text{com}_i^* \leftarrow g^{2\tilde{\sigma}_i}/h$, and samples $c_i \leftarrow \mathbb{Z}_p$ for all $i \in [n]$.
 - * It computes $R_i^* \leftarrow \text{com}_i^*/(\text{pk})^{c_i}$ for all $i \in [n]$.
 - * If, for any i , $(R_i^*, m_i) \in H$, $\mathcal{B}_2$ aborts. Otherwise, it sets $H(R_i^*, m_i) = c_i$ for all $i \in [n]$.
 - * It sets $\mathbf{A}[m_i] \leftarrow \tilde{\sigma}_i$ for all $i \in [n]$.
 - * Lastly, it sends $\{\tilde{\sigma}_i\}_{i \in [n]}$, $\text{aux} = \{R_i^*\}_{i \in [n]}$ and com_k^* to $\mathcal{A}$.
- $\text{EXECUTE}(j, \text{com}_k)$. As in the signer fairness game, if $j \notin \mathcal{K}$ or if $j \neq i^*$ and $\text{com}_k \neq g^{\mathcal{K}[j]}$, or if $j = i^*$ and $\text{com}_k \neq h$, then, $\mathcal{B}_2$ returns $\perp$. Otherwise, if $j = i^*$ and $\text{com}_k = h$, $\mathcal{B}_2$ aborts. Otherwise, $\mathcal{B}_2$ adds j to $\mathbf{X}$ and simply returns $\mathcal{K}[j]$.

Eventually, $\mathcal{A}$ outputs a list of message - signature pairs, $\mathcal{L} = \{m_i^*, \sigma_i^*\}_{i \in [t']}$. $\mathcal{B}_2$ finds a message-signature pair $(m^*, (R^*, s^*)) \in \mathcal{L}$ such that, $m^* \in \mathbf{M}$, the corresponding nonce value R^* is equal to $\mathbf{R}[m^*]$, and, $\mathbf{B}[m^*]$ is not in $\mathbf{X}$. It aborts if it cannot find any such message, or if $\mathbf{B}[m^*] \neq i^*$. Otherwise, let $\tilde{\sigma}^* \leftarrow \mathbf{A}[m^*]$. $\mathcal{B}_2$ sends $2\tilde{\sigma}^* - s^*$ to its challenger. We argue that if $\mathcal{A}$ wins its game and if $\mathcal{B}_2$ does not abort, then $\mathcal{B}_2$ wins its discrete log game. To see this, we note that, $(m^*, (R^*, s^*))$ must be a valid signature for $\mathcal{A}$ to win its game. This means that,

$$g^{s^*} = R^* \cdot (\text{pk})^{c^*}$$

where $c^* = H(R^*, m^*)$. Next, if $\mathcal{B}_2$ does not abort, then the message m^* must belong to batch number i^* and $R^* = R[m^*]$. This means that, R^* must be equal to $g^{2\tilde{\sigma}^*} / (h \cdot (pk)^{c^*})$, where $\tilde{\sigma}^* = A[m^*]$, and $H(R^*, m^*) = c^*$ was sampled $\mathcal{B}_2$ was responding to the SIGN query for $c = i^*$. Combining this with the above equation, we see that,

$$g^{s^*} = \frac{g^{2\tilde{\sigma}^*}}{h \cdot pk^{c^*}} \cdot pk^{c^*} = \frac{g^{2\tilde{\sigma}^*}}{h}$$

The above implies that $h = g^{2\tilde{\sigma}^* - s^*}$, meaning that $\mathcal{B}_2$ indeed computes the correct discrete log, and hence wins its game.

We now analyse the probability of $\mathcal{B}_2$ not aborting. Let E_g denote the event that $\mathcal{B}_2$ correctly guessed the batch number i^* for the forgery message m^* . Since $\mathcal{B}_2$ samples i^* from $[q_s]$ uniformly at random, we have that $\Pr[E_g] = 1/q_s$. Next, for every $i \in [n]$, let $E_{r,i}$ be the event that $\mathcal{B}_2$ aborts when answering a SIGN query because (R_i^*, m_i) was already in the map H – in other words, $\mathcal{A}$ had queried the random oracle on these inputs. Since c_i is sampled uniformly randomly for all $i \in [n]$, R_i^* is a uniformly random element, from the view of the adversary. This means, that $\Pr[E_{r,i}] \leq q_H/p$, where q_H is an upper bound on the number of random oracle queries by $\mathcal{A}$. Then, we get that,

$$\begin{aligned} \text{Adv}_{\mathcal{B}_2}^{\text{dl}}(\lambda) &= \Pr[\text{SignerFairness}_{\mathcal{A}}^{\text{FSE}, \Pi}(1^\lambda) = 1 \wedge E_2 \wedge E_g \wedge_{i \in [n]} \neg E_{r,i}] \\ &\geq \Pr[\text{SignerFairness}_{\mathcal{A}}^{\text{FSE}, \Pi}(1^\lambda) = 1 \wedge E_2 \wedge E_g] - \Pr[\bigvee_{i \in [n]} E_{r,i}] \\ &\geq \frac{1}{q_s} \Pr[\text{SignerFairness}_{\mathcal{A}}^{\text{FSE}, \Pi}(1^\lambda) = 1 \wedge E_2] - \frac{n \cdot q_H}{p} \end{aligned}$$

A.2 Client fairness

Proof of Theorem 3. Assume there are indices $i \in [c]$ and $j \in [n]$ such that $\mathcal{B}[i] = 1$ and $(m, \sigma) := \mathcal{F}[j]$ with $\Pi.\text{Verify}(\text{pk}, m, \sigma) = 0$. Let $(\{m_i\}_{i \in [n]}, \{\tilde{\sigma}_i\}_{i \in [n]}, \text{aux}, \text{com}_k) := \mathcal{I}[c]$, where according to the definition, $m_j = m$. Given that $\mathcal{B}[i] = 1$, it follows that:

$$\begin{aligned} \text{FSE.PVerify}(\{m_i\}_{i \in [n]}, \{\tilde{\sigma}_i\}_{i \in [n]}, \text{aux}, \text{com}_k) &= 1 \\ \iff g^{2\tilde{\sigma}_i} &= \text{com}_k \cdot R_i \cdot \text{pk}^{c_i} \end{aligned}$$

Additionally, we know that $g^k = \text{com}_k$; otherwise, $\mathcal{F}[c] = \perp$. Now, assume for the sake of contradiction that there exists some i such that $\Pi.\text{Verify}(\text{pk}, m_i, \sigma_i) = 0$. As shown in Figure ??, this implies that $g^{s_i} \neq R_i \cdot \text{pk}^{c_i}$, where $s_i = 2\tilde{\sigma}_i - k$. However, we also have:

$$\begin{aligned} R_i \cdot \text{pk}^{c_i} &= g^{2\tilde{\sigma}_i} \cdot \text{com}_k^{-1} \\ &= g^{2\tilde{\sigma}_i - k} \\ &= g^{s_i} \end{aligned}$$

This directly contradicts the assumption that $\Pi.\text{Verify}(\text{pk}, m_i, \sigma_i) = 0$, thereby completing the proof.

B Batch Adaptor Signatures

Definition 7 (Pre-signature correctness for batch adaptor signature). A batch adaptor signature BAS defined on signature scheme Π satisfies pre-signature correctness, if for all messages $m_i \in \{0, 1\}^*$:

$$\Pr \left[\begin{array}{l} \text{BAS.BVerify}(\text{pk}, \{m_i\}_{i \in [n]}, Y, \{\tilde{\sigma}_i\}_{i \in [n]}) = 1 \\ \wedge \Pi.\text{Verify}(\text{pk}, m_i, \sigma_i) = 1 : \forall i \in [n] \\ \wedge (Y, y_i) \in \text{Rel} : \forall i \in [n] \end{array} \middle| \begin{array}{l} (\text{sk}, \text{pk}) \leftarrow \Pi.\text{Gen}(pp), \\ (Y, y) \leftarrow \text{RGen}(1^\lambda), \\ \{\tilde{\sigma}_i\}_{i \in [n]} \leftarrow \text{BAS.pBSign}(\text{sk}, \{m_i\}_{i \in [n]}, Y), \\ \sigma_i := \text{BAS.Adapt}(\text{pk}, \tilde{\sigma}_i, y), \\ y_i := \text{BAS.Extract}(\text{pk}, \tilde{\sigma}_i, \sigma_i, Y) \end{array} \right] = 1.$$

Definition 8 (Pre-signature adaptability for batch adaptor signature). A batch adaptor signature scheme BAS satisfies presignature adaptability, if for all $\lambda \in \mathbb{N}$, messages $m \in \{0, 1\}^*$, statement/witness pairs $(Y, y) \in \text{Rel}$, public key pk , and pre-signatures $[\tilde{\sigma}_i]_{i \in [n]}$, we have $\text{BAS.pVerify}(\text{pk}, [m_i]_{i \in [n]}, Y, [\tilde{\sigma}_i]_{i \in [n]}) = 1$, then for all $i \in [n]$ we have:

$$\Pi.\text{Verify}(\text{pk}, m_i, \text{BAS.Adapt}(\text{pk}, \sigma_i, y)) = 1.$$

Definition 9 (aEUF-CMA security). A batch adaptor signature scheme BAS is unforgeable (or aEUF-CMA secure) if for every PPT adversary $\mathcal{A}$, we have:

$$\Pr [\text{aSigForge}_{\mathcal{A}, \text{BAS}}(\lambda) = 1] \leq \text{negl}(\lambda),$$

where the definition of the experiment $\text{aSigForge}_{\mathcal{A}, \text{AS}}$ is given in defined in Figure 11 and the probability is taken over the random choices of all probabilistic algorithms.

Definition 10 (Witness extractability). A batch adaptor signature scheme BAS is witness extractable if for every PPT adversary $\mathcal{A}$, we have:

$$\Pr [\text{aWitExt}_{\mathcal{A}, \text{AS}}(\lambda) = 1] \leq \text{negl}(\lambda),$$

where the experiment $\text{aWitExt}_{\mathcal{A}, \text{AS}}(\lambda)$ is defined in Figure 12 and the probability is taken over the random choices of all probabilistic algorithms.

Definition 11 (Secure adaptor signature scheme). A batch adaptor signature scheme BAS is secure if it is unforgeable and witness extractable.

$\text{aSigForge}_{\mathcal{A}, \text{BAS}}(\lambda)$	$\text{SIGN}(sk, m)$
1 : $\mathcal{Q} := \phi, (\text{sk}, \text{pk}) \leftarrow \Pi.\text{Gen}(1^\lambda)$	1 : $\sigma \leftarrow \Pi.\text{Sign}(\text{sk}, m)$
2 : $m^* \leftarrow \mathcal{A}^{\text{SIGN}(\text{sk}, \cdot), \text{pSIGN}(\text{sk}, \cdot, \cdot)}(\text{pk})$	2 : $\mathcal{Q} := \mathcal{Q} \cup \{m\}$
3 : $(Y, y) \leftarrow \text{RGen}(1^\lambda)$	3 : return σ
4 : $\tilde{\sigma} \leftarrow \text{BAS.pSign}(\text{sk}, m^*, Y)$	$\text{pSIGN}(\text{sk}, \{m_i\}_{i \in [n]}, Y)$
5 : $\sigma^* \leftarrow \mathcal{A}^{\text{SIGN}(\text{sk}, \cdot), \text{pSIGN}(\text{sk}, \cdot, \cdot)}(\tilde{\sigma}, Y)$	1 : $\tilde{\sigma} \leftarrow \text{BAS.pSign}(\text{sk}, \{m_i\}_{i \in [n]}, Y)$
6 : return $(m^* \notin \mathcal{Q} \wedge \Pi.\text{Verify}(\text{pk}, m^*, \sigma^*))$	2 : $\mathcal{Q} := \mathcal{Q} \cup \{m_i\}_{i \in [n]}$
	3 : return $\tilde{\sigma}$

Fig. 11: Unforgeability game for a batch adaptor signature scheme.

$\text{aWitExt}_{\mathcal{A}, \text{BAS}}(\lambda)$	$\text{SIGN}(\text{sk}, m)$
1 : $\mathcal{Q} := \phi, (\text{sk}, \text{pk}) \leftarrow \Pi.\text{Gen}(1^\lambda)$	1 : $\sigma \leftarrow \Pi.\text{Sign}(\text{sk}, m)$
2 : $(m^*, Y^*) \leftarrow \mathcal{A}^{\text{SIGN}(\text{sk}, \cdot), \text{pSIGN}(\text{sk}, \cdot, \cdot)}(\text{pk})$	2 : $\mathcal{Q} := \mathcal{Q} \cup \{m\}$
3 : $\tilde{\sigma} \leftarrow \text{BAS.pSign}(\text{sk}, m^*, Y^*)$	3 : return σ
4 : $\sigma^* \leftarrow \mathcal{A}^{\text{SIGN}(\text{sk}, \cdot), \text{pSIGN}(\text{sk}, \cdot, \cdot)}(\tilde{\sigma}, Y)$	$\text{pSIGN}(\text{sk}, \{m_i\}_{i \in [n]}, Y)$
5 : $y^* := \text{BAS.Extract}(\text{pk}, \sigma^*, \tilde{\sigma}, Y^*)$	1 : $\tilde{\sigma} \leftarrow \text{BAS.pSign}(\text{sk}, m, Y)$
6 : return $(m^* \notin \mathcal{Q} \wedge (Y^*, y^*) \notin \text{Rel} \wedge \neg \Pi.\text{Verify}(\text{pk}, m^*, \sigma^*))$	2 : $\mathcal{Q} := \mathcal{Q} \cup \{m_i\}_{i \in [n]}$
	3 : return $\tilde{\sigma}$

Fig. 12: Witness extractability game for a batch adaptor signature scheme.

Theorem 4. *BAS construction presented in Figure 8 satisfies pre-signature correctness for batch adaptor signatures.*

Proof. Let $(sk, pk) \leftarrow \Pi.\text{Gen}()$ be the key pair, where $sk \in \mathbb{Z}_q$ is the secret key and $pk = g^{sk}$ is the corresponding public key. The randomness generation algorithm produces $(y, Y) \leftarrow \text{RGen}()$, where $y \in \mathbb{Z}_q$ is chosen uniformly at random and $Y = g^y$. Given a set of messages $\{m_i\}_{i \in [n]}$, the BAS signing algorithm produces intermediate signature values $\{\tilde{\sigma}_i := (s_i, R'_i)\}_{i \in [n]}$ using a random value $r_i \in \mathbb{Z}_q$ for each message m_i . The algorithm computes:

$$R'_i = \frac{g^{r_i}}{Y} = g^{r_i - y}$$

Now it implies

$$c_i = \text{H}(R'_i, m_i) \implies s_i = \frac{r_i + c_i \cdot sk}{2},$$

In the adaptation step, the adapted signature is:

$$s'_i = 2s_i - y = r_i + c_i \cdot sk - y.$$

In the extraction step, the extracted value y_i is computed as:

$$y_i = 2\tilde{s}_i - s_i \implies y_i = y.$$

Thus, the relation $(Y, y_i) \in \text{Rel}$ holds as $Y = g^{y_i}$. The BAS batch verification algorithm BVerify checks:

$$g^{2s_i} \stackrel{?}{=} Y \cdot R'_i \cdot \text{pk}^{c_i} \xrightarrow{R'_i = g^{r_i - y}} g^{2s_i} \stackrel{?}{=} g^y \cdot g^{r_i - y} \cdot g^{c_i \cdot sk} = g^{r_i + c_i \cdot sk}.$$

Since $s_i = \frac{r_i + c_i \cdot sk}{2}$, the equality holds which ensures BAS.BVerify accepts. Similarly, Schnorr verification checks: $g^{s'_i} \stackrel{?}{=} R'_i \cdot \text{pk}^{c_i}$. Expanding $R'_i = g^{r_i - y}$, we get:

$$g^{s'_i} \stackrel{?}{=} g^{r_i - y} \cdot g^{c_i \cdot sk} \xrightarrow{s'_i = r_i + c_i \cdot sk} g^{s'_i} = g^{r_i + c_i \cdot sk - y},$$

which holds, ensuring $\Pi.\text{Verify}(\text{pk}, m, \sigma) = 1$. Thus, we conclude that pre-signature correctness for BAS holds.

Theorem 5. *BAS construction presented in Figure 8 satisfies pre-signature adaptability for batch adaptor signatures.*

Proof. Given pre-signatures $\tilde{\sigma}_i = (s_i, R'_i)$ and satisfying pair (y, Y) , i.e. $Y = g^y$, such that the partial verification holds, in other words:

$$\text{BAS.pVerify}(\text{pk}, [m_i]_{i \in [n]}, Y, [\tilde{\sigma}_i]_{i \in [n]}) = 1$$

Which implies that defining $c_i = H(R'_i, m_i)$ we have:

$$i \in [n] : g^{2s_i} = Y \cdot R'_i \cdot \text{pk}^{c_i}$$

Running Adaptation gives us:

$$\sigma = \text{BAS.Adapt}(\text{pk}, \sigma_i, y) \implies s'_i = 2 \cdot s_i - y$$

Now replacing s'_i with s_i in equation above gives us:

$$g^{s'_i + y} = Y \cdot R'_i \cdot \text{pk}^{c_i} \implies g^{s'_i} = R'_i \cdot \text{pk}^{c_i}$$

Which is the Schnorr verification which implies that for all $i \in [n]$:

$$\Pi.\text{Verify}(\text{pk}, m_i, \text{BAS.Adapt}(\text{pk}, \sigma_i, y)) = 1$$